



GNU Compiler Collection (gcc)

Pasos de Compilación, Visualización Intermedia y Optimización

Del Preprocesamiento al Código Ejecutable usando GCC

Resumen: Esta guía tiene como objetivo principal enseñar a los estudiantes de ingeniería a desglosar el proceso completo de compilación con GCC, desde el código fuente en C hasta el ejecutable final. A través de ejemplos prácticos reproducibles, mediciones cuantitativas y visualizaciones, el lector aprenderá a inspeccionar cada etapa intermedia (preprocesado, ensamblador, objeto), comprender el impacto real de diferentes niveles de optimización (-O0 a -O3, -Os) en el tamaño del código y el tiempo de ejecución, y aplicar mejores prácticas para proyectos universitarios y profesionales.

Curso: Sistemas Operativos

German Sánchez Torres, Ph.D.

Versión: 1.0

Índice general

1	Introducción	5
1.1	Propósito y alcance	5
1.2	El lector y los conocimientos de partida	6
1.3	Una forma de estudiar la herramienta	6
1.4	Recorrido de la guía	6
1.5	Convenciones de lectura y experimentación	7
1.6	Fuentes y criterio técnico	7
2	Fundamentos de GCC	9
2.1	GCC como sistema de compilación	9
2.2	Reconocer el entorno de trabajo	10
2.3	Del primer programa al contrato de compilación	10
2.4	Compilación separada y organización del programa	11
2.5	Una idea que organiza el resto de la guía	11
3	Del código fuente al ejecutable	13
3.1	El mapa general de la traducción	13
3.2	Preprocesamiento: construir la unidad de traducción	14
3.3	Compilación: comprender y transformar el programa	14
3.4	Ensamblado: crear un objeto reubicable	15
3.5	Enlace: formar un programa coherente	15
3.6	De la receta al modelo mental	16
4	Optimización y evaluación del rendimiento	17
4.1	Qué puede transformar el compilador	17
4.2	Los niveles como políticas, no como escalones absolutos	18
4.3	Medir es formular un experimento	18
4.4	Un banco de pruebas pequeño pero observable	19
4.5	Elegir con evidencia	20

Índice de figuras

2.1 Separación conceptual entre frontal, optimización intermedia y generación de código.	9
3.1 Productos observables de la cadena de compilación.	13

Índice de cuadros

3.1 La etapa adecuada según la pregunta de diagnóstico	16
4.1 Lectura práctica de los niveles de optimización	18
4.2 Registro que debe completarse con resultados observados	19

Capítulo 1

Introducción

Cuando se ejecuta `gcc programa.c -o programa`, la brevedad del comando puede ocultar la cantidad de trabajo que ocurre detrás. El archivo escrito por el programador debe convertirse primero en una unidad de traducción, después en representaciones internas que el compilador pueda analizar, más tarde en instrucciones para una arquitectura concreta y, finalmente, en un ejecutable cuyas referencias externas hayan sido resueltas. Compilar no consiste, por tanto, en sustituir palabras de C por instrucciones de máquina. Es una cadena de transformaciones que preserva el significado observable del programa mientras cambia radicalmente su representación [1, 4].

Esta guía estudia esa cadena desde la perspectiva de quien programa sistemas. No pretende enseñar a construir un compilador completo, pero sí abrir la “caja negra” lo suficiente para responder preguntas prácticas: ¿por qué una macro produce un error difícil de interpretar?, ¿qué información contiene un archivo objeto?, ¿de dónde surge una referencia indefinida durante el enlace?, ¿por qué dos niveles de optimización generan ensamblador distinto? Esta perspectiva conecta el código fuente con la arquitectura, el sistema operativo y las herramientas de desarrollo, una relación central en la formación de sistemas [3].

1.1 Propósito y alcance

El propósito es que el lector desarrolle un modelo mental del proceso de compilación y aprenda a contrastarlo con evidencia. Al terminar, debería poder seguir un programa sencillo desde su archivo `.c` hasta el ejecutable, detener el proceso en etapas intermedias e interpretar los artefactos producidos. También debería estar en condiciones de formular y comprobar una hipótesis de rendimiento sin asumir que una bandera más agresiva garantiza un programa más rápido.

El documento se concentra en C sobre sistemas tipo Unix y utiliza GCC como hilo conductor. Muchos principios son generales —análisis, representaciones intermedias, generación de código, ensamblado y enlace—, pero los nombres de herramientas, formatos y convenciones pueden variar entre plataformas. Cuando se muestra ensamblador x86-64 o un objeto ELF, se presenta un caso de estudio, no una propiedad universal de todos los sistemas.

El objetivo no es memorizar opciones de línea de comandos. Es comprender qué transformación solicita cada opción, qué artefacto produce y qué información puede obtenerse de ese artefacto.

1.2 El lector y los conocimientos de partida

La guía está pensada principalmente para estudiantes que ya escriben programas pequeños en C y desean entender qué ocurre entre el código fuente y la ejecución. También puede servir a desarrolladores que necesiten interpretar diagnósticos del compilador, inspeccionar binarios o diseñar experimentos de rendimiento. No se presupone experiencia previa en construcción de compiladores ni dominio del lenguaje ensamblador.

Conviene reconocer funciones, estructuras de control, arreglos y punteros; saber ejecutar comandos en una terminal; y distinguir, al menos de manera intuitiva, entre código fuente, biblioteca y programa ejecutable. Los conceptos de arquitectura necesarios se introducen en contexto. Por ejemplo, los registros aparecen cuando se estudia una función compilada, y las secciones ELF cuando se abre un archivo objeto. Así, cada término responde a una necesidad concreta antes de convertirse en vocabulario técnico.

El entorno más directo es una distribución GNU/Linux con GCC y GNU Binutils. WSL ofrece una experiencia semejante en Windows. En macOS debe comprobarse qué compilador responde al nombre `gcc`, pues con frecuencia ese comando invoca Clang. La guía evita fijar una versión mínima: las opciones esenciales son estables, mientras que el conjunto exacto de optimizaciones y arquitecturas soportadas cambia con cada publicación del compilador [7].

1.3 Una forma de estudiar la herramienta

Cada tema se aborda mediante un ciclo breve: leer una explicación, producir un artefacto e inspeccionarlo. El texto introduce primero la idea que da sentido al comando; el comando genera luego una salida observable; y esa salida permite verificar o corregir la explicación. Esta dinámica evita que la terminal se convierta en una colección de recetas sin conexión conceptual.

Supóngase, por ejemplo, que se afirma que el preprocesador expande una macro. La actividad no termina al ejecutar `gcc -E`: hay que localizar la expresión transformada, compararla con el original y explicar por qué los paréntesis de una macro funcional importan. De forma análoga, al comparar `-O0` con `-O2` no basta contar líneas de ensamblador. Es necesario comprobar que ambos ejecutables producen el mismo resultado y registrar las condiciones de medición.

Los ejemplos son deliberadamente pequeños para que una relación causal permanezca visible. Un programa real mezcla cabeceras, bibliotecas, múltiples unidades de traducción y numerosas decisiones de optimización; comenzar por un caso reducido permite aislar una transformación antes de regresar a esa complejidad. Los tratados de compiladores utilizan la misma estrategia: separar las fases conceptuales aunque una implementación industrial pueda combinarlas o repetirlas internamente [1, 4].

Conserve los comandos, la versión de GCC y las salidas relevantes de cada práctica. Un cuaderno de laboratorio convierte una demostración aislada en un resultado reproducible.

1.4 Recorrido de la guía

El Capítulo 2 presenta GCC como un controlador de herramientas y explica la relación entre el compilador, el ensamblador y el enlazador. Allí se construye el primer programa y se introduce

la compilación separada de varios archivos.

El Capítulo 3 sigue con detalle los productos de la traducción: salida preprocesada, ensamblador, objeto relocatable y ejecutable. Las utilidades `objdump`, `nm` y `readelf` se incorporan cuando permiten responder una pregunta específica sobre esos productos.

El Capítulo 4 cambia el foco desde la forma del artefacto hacia la calidad del código generado. Explica los niveles de optimización como conjuntos de decisiones, no como promesas de velocidad, y propone un protocolo de medición que separa datos observados de expectativas.

Este recorrido forma una sola narración: primero se reconoce quién coordina la construcción; después se observa cómo cambia el programa; por último se evalúa cómo esas transformaciones afectan el resultado y su desempeño.

1.5 Convenciones de lectura y experimentación

Los comandos, nombres de archivo y fragmentos literales aparecen en tipografía monoespaciada. Las cajas azules fijan una idea que conviene conservar; las verdes amplían el contexto; las naranjas advierten sobre supuestos o riesgos; y las prácticas proponen una comprobación. Una opción como `-O2` se escribe con el guion que recibe la terminal, mientras que una extensión como `.o` identifica una clase de archivo, no una obligación del estándar.

Las salidas mostradas son representativas. Direcciones, nombres internos, registros y orden de instrucciones pueden cambiar con la versión de GCC, la arquitectura objetivo, el sistema operativo y las opciones activas. Esa variabilidad no es un defecto del experimento: es precisamente una razón para registrar el entorno y evitar conclusiones universales a partir de una sola compilación.

1.6 Fuentes y criterio técnico

La explicación se apoya en textos universitarios de compiladores y sistemas, en especificaciones de formatos e interfaces y en la documentación mantenida por los proyectos GNU. Las fuentes cumplen funciones diferentes. Aho y colaboradores, así como Cooper y Torczon, proporcionan el marco conceptual de las fases y representaciones [1, 4]; Bryant y O'Hallaron conectan el código con la máquina y el enlace [3]; Levine profundiza en resolución de símbolos y relocación [11]; y los manuales de GCC y Binutils precisan el comportamiento de las opciones y herramientas [5, 7].

Esta jerarquía evita dos errores frecuentes: convertir una observación particular en regla general y tratar una opción de implementación como si perteneciera al lenguaje C. El estándar define el programa y sus requisitos semánticos; el compilador decide cómo traducirlo para un objetivo concreto, siempre que conserve el comportamiento exigido para programas conformes [9].

Una referencia sería no reemplaza la observación del sistema real. La especificación establece qué debe cumplirse; el experimento muestra cómo una versión y una plataforma concretas lo materializan.

Capítulo 2

Fundamentos de GCC

2.1 GCC como sistema de compilación

Cuando se ejecuta `gcc programa.c -o programa`, parece que una sola herramienta transforma el código fuente en un ejecutable. En realidad, el comando `gcc` actúa como un *controlador*: interpreta las opciones, reconoce el lenguaje y coordina el preprocesador, el compilador, el ensamblador y el enlazador. Esta distinción es importante porque permite entender tanto los archivos intermedios como el origen de los errores.

El nombre GCC designó inicialmente al *GNU C Compiler*; hoy corresponde a la *GNU Compiler Collection*, una colección con frontales para varios lenguajes y soportes para numerosas arquitecturas. Su diseño separa el análisis propio de cada lenguaje de las transformaciones comunes y de la generación de código específica de cada procesador [6, 14]. El ensamblador, el enlazador y utilidades como `nm`, `readelf` y `objdump` pertenecen, por su parte, a GNU Binutils [5]. En una instalación habitual ambas familias de programas cooperan, pero no son la misma pieza de software.



Figura 2.1: Separación conceptual entre frontal, optimización intermedia y generación de código.

El frontal comprueba la sintaxis y la semántica del lenguaje. Después, GCC expresa el programa mediante representaciones internas, entre ellas GIMPLE, adecuada para muchas optimizaciones, y RTL (*Register Transfer Language*), cercana a las operaciones de la máquina. El *backend* traduce esas representaciones a instrucciones de la arquitectura elegida [6]. Este recorrido explica por qué una misma fuente puede producir ejecutables distintos al cambiar el procesador objetivo o las opciones, aun cuando conserve su comportamiento observable.

GCC no es solamente un traductor de texto C a código máquina. Es un sistema que coordina etapas y reutiliza representaciones intermedias para separar el lenguaje fuente de la arquitectura de destino.

2.2 Reconocer el entorno de trabajo

La forma de instalar GCC depende del sistema operativo y de su administrador de paquetes. Por eso, una guía duradera no necesita fijar una versión supuestamente «recomendada»; necesita enseñar a identificar el compilador que realmente se está usando. Tras instalar el paquete disponible para la plataforma, conviene registrar estos comandos:

```
gcc --version
gcc -dumpmachine
gcc -v
```

Código 2.1: Identificación del compilador y del sistema objetivo

El primero informa la versión; el segundo muestra la terna de destino, y el tercero expone detalles de configuración y rutas de búsqueda. Estos datos forman parte de cualquier experimento reproducible. Si se desea observar la secuencia de programas que el controlador ejecutaría sin ponerla en marcha, puede añadirse `###`:

```
gcc -### programa.c -o programa
```

Código 2.2: Inspección de los comandos coordinados por GCC

En GNU/Linux, GCC suele instalarse desde los repositorios de la distribución. En macOS puede emplearse una distribución de GCC instalada mediante un gestor de paquetes; el compilador llamado `gcc` por las herramientas del sistema puede ser, en realidad, Clang. En Windows son comunes entornos como MSYS2 o WSL. La comprobación anterior evita inferir la implementación a partir del nombre del comando.

2.3 Del primer programa al contrato de compilación

Considérese un programa mínimo. Aunque es breve, ya contiene los elementos esenciales: una directiva que incorpora una interfaz de biblioteca, la función que inicia la ejecución y una llamada cuya implementación se resolverá durante el enlace.

```
1 #include <stdio.h>
2
3 int main(void) {
4     puts("Hola, mundo");
5     return 0;
6 }
```

Código 2.3: Programa mínimo en C

Si el archivo se llama `hola.c`, el comando siguiente solicita un ejecutable llamado `hola`. La opción `-std=c17` declara el dialecto del lenguaje que se desea aceptar; `-Wall` y `-Wextra` habilitan grupos amplios de diagnósticos, y `-pedantic` pide advertencias asociadas con el cumplimiento estricto del estándar elegido [7, 9].

```
gcc -std=c17 -Wall -Wextra -pedantic hola.c -o hola
./hola
```

Código 2.4: Compilación con un contrato explícito

Los nombres de estas opciones pueden inducir a error. `-Wall` no activa literalmente todas las advertencias disponibles: selecciona un conjunto que el proyecto GCC considera especialmente útil. `-Wextra` lo amplía. Convertirlas en errores mediante `-Werror` puede ser apropiado en integración continua, pero exige una política consciente, pues una actualización del compilador puede incorporar nuevos diagnósticos.

Las opciones describen un contrato de construcción. `-Iruta` añade directorios de cabeceras; `-Lruta` añade directorios de bibliotecas; `-lnombre` solicita una biblioteca; `-g` incorpora información para el depurador, y `-O0`, `-Og`, `-O2` u otros niveles establecen políticas de optimización. Su significado se estudia en el Capítulo 4.

Una compilación sin mensajes no demuestra que el programa sea correcto. El compilador comprueba reglas del lenguaje y puede advertir patrones sospechosos, pero no conoce todas las intenciones del programador ni sustituye las pruebas.

2.4 Compilación separada y organización del programa

Un programa real suele repartirse en varios archivos. Las cabeceras declaran la interfaz que comparten distintas unidades de traducción; los archivos `.c` contienen las definiciones. El preprocesamiento de cada fuente produce una unidad de traducción que se compila por separado. Finalmente, el enlazador reúne los objetos y resuelve las referencias entre ellos [3, 11].

Por ejemplo, una función puede declararse en `operaciones.h`, definirse en `operaciones.c` y utilizarse desde `main.c`. La construcción explícita deja visible la frontera entre compilación y enlace:

```
gcc -std=c17 -Wall -Wextra -c operaciones.c -o operaciones.o
gcc -std=c17 -Wall -Wextra -c main.c -o main.o
gcc main.o operaciones.o -o calculadora
```

Código 2.5: Construcción de un programa formado por varias unidades

La opción `-c` detiene el proceso después de crear cada archivo objeto. Esto permite recompilar solamente las unidades que cambiaron, principio que aprovechan herramientas de construcción como `make`. También obliga a distinguir dos clases de problema: un error de compilación pertenece a una unidad concreta; un error de enlace aparece cuando las unidades ya fueron traducidas, pero falta una definición o existen definiciones incompatibles.

Cree dos archivos fuente y una cabecera. Compile cada fuente con `-c`; inspeccione los objetos con `nm`; enlace el ejecutable y, por último, modifique una sola implementación. Observe qué producto necesita regenerarse y cuál puede conservarse.

2.5 Una idea que organiza el resto de la guía

Usar GCC con criterio consiste en poder responder tres preguntas: qué lenguaje se prometió aceptar, qué transformación se solicitó y en qué etapa surgió el resultado o el error. El siguiente

capítulo abre esa «caja negra» y sigue un programa desde el texto preprocesado hasta el ejecutable.

Capítulo 3

Del código fuente al ejecutable

El comando de compilación abrevia una cadena de transformaciones. Para estudiarla, resulta útil detener el proceso en puntos concretos y examinar qué información aparece, desaparece o se reorganiza. La división en cuatro etapas que se presenta aquí es pedagógica: una implementación moderna contiene muchas fases internas adicionales [1, 4].

3.1 El mapa general de la traducción

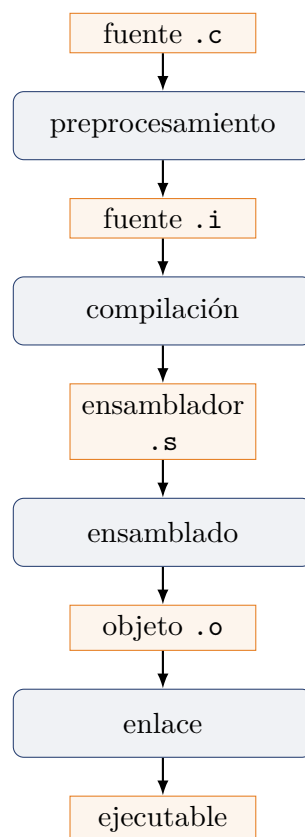


Figura 3.1: Productos observables de la cadena de compilación.

GCC permite conservar productos intermedios con `-save-temps` o detenerse con `-E`, `-S` y `-c`. No se trata solo de opciones operativas: cada una ofrece una perspectiva distinta del programa.

3.2 Preprocesamiento: construir la unidad de traducción

Antes de analizar el lenguaje C, el preprocesador atiende las directivas que comienzan con `#`. Expande macros, incluye el contenido indicado por las cabeceras y selecciona regiones mediante compilación condicional. El resultado ya no conserva la misma organización textual que escribió el programador, pero constituye la entrada que recibirá el compilador según el modelo de traducción de C [7, 9].

```
1 #include <stdio.h>
2 #define CUADRADO(x) ((x) * (x))
3
4 int main(void) {
5     printf("%d\n", CUADRADO(5));
6 }
```

Código 3.1: Una macro visible antes del análisis de C

El comando `gcc -E ejemplo.c -o ejemplo.i` permite inspeccionar la expansión. El archivo será extenso porque incorpora declaraciones de las cabeceras. Buscar la función `main` y la expresión expandida suele ser más instructivo que leerlo de principio a fin.

Las macros son sustituciones de tokens, no funciones. Por eso, `CUADRADO(i++)` evaluaría el incremento más de una vez. Los paréntesis reducen errores de precedencia, pero no cambian esa propiedad. Cuando se requiere evaluación única y comprobación de tipos, una función suele expresar mejor la intención.

3.3 Compilación: comprender y transformar el programa

Con la unidad de traducción preparada, el frontal reconoce tokens, estructura sintáctica y reglas semánticas. GCC convierte después el programa a representaciones intermedias sobre las que realiza análisis y transformaciones. GIMPLE simplifica muchas construcciones del lenguaje; RTL modela operaciones próximas a registros e instrucciones. Esta arquitectura permite compartir optimizaciones entre lenguajes y separar buena parte de ellas de la máquina de destino [6, 14].

La orden siguiente detiene el proceso al generar ensamblador:

```
gcc -std=c17 -O0 -S suma.c -o suma.s
```

Código 3.2: Generación de ensamblador legible

Para una función como la siguiente, el archivo `.s` mostrará cómo se materializan parámetros, variables, saltos y retornos.

```
1 int sumar(const int *datos, int n) {
2     int total = 0;
3     for (int i = 0; i < n; ++i)
4         total += datos[i];
5     return total;
6 }
```

Código 3.3: Función cuyo recorrido puede estudiarse en ensamblador

El ensamblador exacto no es una traducción universal del código C. Depende de la arquitectura objetivo, la interfaz binaria de aplicaciones (ABI), la versión del compilador y sus opciones. En x86-64, la ABI determina, entre otras cosas, dónde llegan los argumentos y cómo se preservan ciertos registros [12]; el manual de Intel documenta el significado de las instrucciones [8]. Incluso la notación puede variar: GCC emite normalmente sintaxis AT&T, aunque `-masm=intel` solicita sintaxis Intel en los objetivos compatibles.

Leer ensamblador no significa memorizar todas las instrucciones. Conviene comenzar identificando la entrada de datos, las comparaciones, los saltos del bucle, los accesos a memoria y el valor de retorno.

3.4 Ensamblado: crear un objeto reubicable

El ensamblador convierte las instrucciones textuales en código máquina y metadatos. Con `gcc -c suma.s -o suma.o` se obtiene un objeto, pero todavía no un programa ejecutable completo. En sistemas GNU/Linux es habitual el formato ELF: el archivo organiza secciones de código y datos, una tabla de símbolos y entradas de reubicación que indican qué direcciones deberán ajustarse más adelante [3, 12].

Las utilidades de GNU Binutils permiten observar esas capas sin modificar el archivo:

```
nm suma.o
readelf -S -s -r suma.o
objdump -d suma.o
```

Código 3.4: Tres vistas complementarias de un archivo objeto

`nm` resume símbolos definidos y pendientes; `readelf` expone la estructura ELF, y `objdump -d` desensambla los bytes de las secciones ejecutables [5]. Un símbolo marcado como indefinido no constituye necesariamente un error: puede corresponder a una función que proporcionará otra unidad o una biblioteca durante el enlace.

3.5 Enlace: formar un programa coherente

El enlazador combina objetos y bibliotecas. Primero resuelve referencias simbólicas: relaciona cada uso externo con una definición compatible. Después aplica reubicaciones y construye la imagen que cargará el sistema. Esta etapa explica mensajes como *undefined reference* y *multiple definition*; ya no son errores de sintaxis, sino de composición del programa [3, 11].

```
gcc main.o suma.o -o programa
```

Código 3.5: Enlace mediante el controlador de GCC

Es preferible invocar el enlace mediante `gcc` en lugar de llamar directamente a `ld`. El controlador incorpora los archivos de inicio, las rutas y las bibliotecas de soporte apropiadas para el lenguaje y el sistema. Según la plataforma, el resultado puede depender de bibliotecas compartidas que el cargador localizará al iniciar el proceso; `readelf -d programa` permite estudiar esa información.

Tabla 3.1: La etapa adecuada según la pregunta de diagnóstico

Detención	Producto	Pregunta principal
-E	fuelle preprocesada	¿Qué incluyeron o expandieron las directivas?
-S	ensamblador	¿Qué código produjo el compilador?
-c	objeto reubicable	¿Qué símbolos y reubicaciones quedaron pendientes?
sin detención	ejecutable	¿Se resolvieron correctamente objetos y bibliotecas?

3.6 De la receta al modelo mental

La utilidad de separar las etapas no consiste en ejecutar cuatro comandos en lugar de uno. Consiste en localizar la frontera donde cambia la información. Si una macro produce una expresión inesperada, se inspecciona el preprocesado; si interesa una transformación, se compara el ensamblador; si falta una función, se estudian símbolos y enlace. Así, la cadena deja de ser una caja negra y se convierte en un método de diagnóstico.

Capítulo 4

Optimización y evaluación del rendimiento

Optimizar no significa «hacer más rápido» mediante una única técnica. El compilador analiza el programa y aplica transformaciones que pueden reducir instrucciones, evitar cálculos redundantes, mejorar el uso de registros o facilitar el paralelismo. El criterio central es preservar el comportamiento observable que el lenguaje exige para los programas definidos [1, 4, 9].

4.1 Qué puede transformar el compilador

Algunas optimizaciones operan en una región pequeña: propagan constantes, eliminan expresiones que no afectan al resultado o simplifican operaciones. Otras necesitan una visión más amplia: mueven cálculos invariantes fuera de un bucle, integran funciones, reorganizan el flujo o vectorizan iteraciones. La teoría y la práctica de estas transformaciones se apoyan en análisis de flujo de datos, dependencias y aliasing [2].

El compilador solo puede razonar a partir del contrato del lenguaje. En C, una operación con comportamiento indefinido no impone un resultado concreto a la implementación. Por ejemplo, el desbordamiento de un entero con signo no está definido por el estándar, mientras que el de un entero sin signo se calcula módulo una potencia de dos [9]. Una optimización puede aprovechar esa diferencia; por ello, que un programa «funcione» con `-O0` no demuestra que sea correcto.

```
1 int incrementar(int x) {  
2     if (x + 1 > x)  
3         return x + 1;  
4     return x;  
5 }
```

Código 4.1: Una condición que no protege frente al desbordamiento con signo

Si `x` vale `INT_MAX`, evaluar `x + 1` ya incurre en comportamiento indefinido. La solución robusta es comprobar antes de operar, por ejemplo `x == INT_MAX`. Opciones como `-fwrapv` modifican supuestos concretos sobre el desbordamiento con signo, pero no corrigen de forma general un programa que viola otras reglas. Los sanitizadores, como `-fsanitize=undefined,address`, son herramientas de diagnóstico y complementan las advertencias y las pruebas.

Las diferencias entre una compilación sin optimización y otra optimizada suelen revelar dependencias no válidas, datos no inicializados, accesos fuera de límites o condiciones de carrera. Desactivar la optimización puede ocultar el síntoma, pero no elimina la causa.

4.2 Los niveles como políticas, no como escalones absolutos

GCC agrupa transformaciones en niveles. `-O0` prioriza una relación directa con el fuente y desactiva la mayoría de optimizaciones; `-Og` busca conservar una experiencia de depuración razonable mientras aplica transformaciones útiles; `-O1` y `-O2` incrementan el trabajo de optimización; `-O3` añade transformaciones más agresivas; `-Os` favorece el tamaño, y `-Ofast` permite optimizaciones que pueden relajar requisitos semánticos, especialmente en aritmética de punto flotante [7].

Tabla 4.1: Lectura práctica de los niveles de optimización

Nivel	Prioridad orientativa	Uso razonable
<code>-O0</code>	transformación mínima	diagnóstico inicial y observación cercana al fuente
<code>-Og</code>	depuración con optimización moderada	desarrollo cuando se desea depurar y evitar código innecesariamente literal
<code>-O2</code>	equilibrio general	construcciones de producción, después de probar el programa
<code>-O3</code>	mayor agresividad	cargas medidas que realmente se benefician
<code>-Os</code>	reducción de tamaño	sistemas con restricciones o sensibilidad a la huella
<code>-Ofast</code>	rendimiento con supuestos relajados	casos controlados cuyo modelo numérico admita esas concesiones

No existe una relación universal en la que cada nivel produzca siempre un ejecutable más rápido o más pequeño que el anterior. El conjunto exacto de opciones depende de la versión y del objetivo; el propio manual recomienda consultar `gcc -Q -help=optimizers` para conocer las transformaciones habilitadas en una configuración concreta [7]. `-O3` puede aumentar el tamaño sin mejorar una carga determinada, y `-Os` puede resultar competitivo cuando un código más compacto aprovecha mejor las cachés. La decisión debe apoyarse en mediciones.

4.3 Medir es formular un experimento

Una comparación válida comienza antes del cronómetro. Debe fijar la fuente, la entrada, el compilador, la arquitectura objetivo y todas las opciones. También debe comprobar que cada ejecutable produce el mismo resultado esperado. Sin esa verificación, una versión aparentemente rápida podría haber omitido trabajo por un error o por una optimización legítima de código cuyo resultado nunca se observa.

El entorno introduce variabilidad: frecuencia dinámica del procesador, calentamiento de cachés, planificación del sistema, procesos vecinos y disposición del ejecutable en memoria. Estudios experimentales han mostrado que cambios aparentemente irrelevantes pueden alterar medidas

y conducir a conclusiones equivocadas [13]. Por ello, no basta con registrar una sola ejecución. Conviene realizar calentamientos, repetir, conservar todos los datos y resumir su distribución; cuando se comparan sistemas complejos, los intervalos de confianza y el diseño jerárquico ayudan a expresar la incertidumbre [10].

Un protocolo mínimo puede construirse así:

```
gcc -std=c17 -Wall -Wextra -O0 banco.c -o banco_00
gcc -std=c17 -Wall -Wextra -Og banco.c -o banco_0g
gcc -std=c17 -Wall -Wextra -O2 banco.c -o banco_02
gcc -std=c17 -Wall -Wextra -O3 banco.c -o banco_03
size banco_00 banco_0g banco_02 banco_03
```

Código 4.2: Construcción controlada de cuatro variantes

Después se ejecuta cada variante varias veces con la misma entrada, se descarta o registra por separado el calentamiento y se almacena cada observación. En sistemas tipo Unix, `/usr/bin/time` puede aportar tiempo de usuario, tiempo de sistema y memoria máxima, pero su salida debe redirigirse a un archivo en vez de copiar solamente el mejor resultado.

Tabla 4.2: Registro que debe completarse con resultados observados

Nivel	Tamaño (bytes)	Mediana (s)	Dispersión	Salida correcta
-00				
-0g				
-02				
-03				

La tabla permanece deliberadamente vacía: las cifras dependen del programa, la entrada y la máquina. Presentar números inventados como resultados «esperados» convertiría una hipótesis en evidencia. Antes de medir, debe registrarse al menos la salida de `gcc -version`, `gcc -dumpmachine`, el sistema, el procesador y el comando completo.

4.4 Un banco de pruebas pequeño pero observable

Para explorar los niveles se necesita una carga suficientemente larga y un resultado que el programa haga observable. El siguiente núcleo calcula una suma de comprobación; el tamaño se recibe como argumento o se fija de manera que cada ejecución dure lo bastante para superar el ruido del temporizador.

```
1 #include <stdint.h>
2 #include <stdio.h>
3
4 int main(void) {
5     uint64_t suma = 0;
6     for (uint64_t i = 0; i < 500000000ULL; ++i)
7         suma += (i * 17U) ^ (i >> 3);
8     printf("%llu\n", (unsigned long long)suma);
9     return 0;
10 }
```

Código 4.3: Núcleo sencillo para un experimento reproducible

El tipo sin signo hace explícita la aritmética modular y la impresión impide que el cálculo completo sea trabajo muerto. Aun así, este programa solo representa una clase de carga. Una conclusión aplicable a un proyecto exige medir sus datos y rutas reales. Para investigar por qué cambia el rendimiento, puede compararse el ensamblador con `-S`, examinar los informes de vectorización y usar un perfilador; el tiempo total por sí solo indica que hubo una diferencia, no cuál transformación la produjo.

Ejecute el banco con cuatro niveles, aleatorice el orden de las repeticiones y conserve los datos crudos. Verifique que todas las variantes imprimen la misma suma. Después compare medianas y dispersión, y contraste las diferencias con el ensamblador generado.

4.5 Elegir con evidencia

Durante el desarrollo, `-Og -g` suele ofrecer un punto de partida útil para depurar; para una entrega, `-O2` es una hipótesis razonable, no una garantía. `-O3`, `-Os` o ajustes específicos de arquitectura solo se justifican cuando una medida representativa muestra una mejora relevante y las pruebas conservan la corrección. Comparar GCC con otro compilador exige el mismo cuidado: mismo programa, objetivo, contrato numérico, bibliotecas y protocolo. La pregunta profesional no es «qué compilador es más rápido», sino «qué configuración produce el mejor resultado verificable para esta carga y estas restricciones».

Referencias

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Pearson, 2nd edition, 2011. ISBN 9780133002140.
- [2] David F. Bacon, Susan L. Graham, and Oliver J. Sharp. Compiler transformations for high-performance computing. *ACM Computing Surveys*, 26(4):345–420, 1994. doi: 10.1145/197405.197406.
- [3] Randal E. Bryant and David R. O’Hallaron. *Computer Systems: A Programmer’s Perspective*. Pearson, 3rd edition, 2016. ISBN 9780134092669.
- [4] Keith D. Cooper and Linda Torczon. *Engineering a Compiler*. Morgan Kaufmann, 3rd edition, 2022. ISBN 9780128154120.
- [5] Free Software Foundation. *GNU Binary Utilities*. GNU Project, binutils 2.46 edition, 2026. URL <https://sourceware.org/binutils/docs/>.
- [6] Free Software Foundation. *GNU Compiler Collection Internals*. GNU Project, gcc 16.1 edition, 2026. URL <https://gcc.gnu.org/onlinedocs/gccint/>.
- [7] Free Software Foundation. *Using the GNU Compiler Collection*. GNU Project, gcc 16.1 edition, 2026. URL <https://gcc.gnu.org/onlinedocs/gcc/>.
- [8] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer’s Manual*. Intel Corporation, 2025.
- [9] ISO/IEC. Information technology—programming languages—c. Technical Report ISO/IEC 9899:2018, International Organization for Standardization, 2018.
- [10] Tomas Kalibera and Richard E. Jones. Rigorous benchmarking in reasonable time. In *Proceedings of the 2013 International Symposium on Memory Management*, pages 63–74. ACM, 2013. doi: 10.1145/2464157.2464160.
- [11] John R. Levine. *Linkers and Loaders*. Morgan Kaufmann, 1999. ISBN 9781558604964.
- [12] Michael Matz, Jan Hubička, Andreas Jaeger, and Mark Mitchell. System v application binary interface: Amd64 architecture processor supplement. Technical Report Draft 0.99.7, System V ABI Working Group, 2014.
- [13] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 265–276. ACM, 2009. doi: 10.1145/1508244.1508275.
- [14] Diego Novillo. GCC: An architectural overview, current status, and future directions. In *Proceedings of the Linux Symposium*, volume 2, pages 193–208, 2006.